

SALESTORM – Phase I: Requirements & Assumptions

1. Problem Statement

SALESTORM is a high-volume e-commerce flash-sale platform where a large number of customers may attempt to purchase a limited-stock product simultaneously.

The system must handle **10,000 concurrent purchase requests for only 100 available units**, while preventing inventory overselling, duplicate reservations and duplicate payments. It must also maintain a correct order lifecycle even when payment, database or downstream services experience failures.

2. Business Objective

The system should:

- Handle sudden spikes in customer traffic without system collapse.
- Prevent inventory overselling.
- Temporarily reserve inventory during checkout.
- Release reservations when payment fails or times out.
- Process payments safely and idempotently.
- Maintain a consistent and traceable order lifecycle.
- Recover gracefully from service, database and payment failures.
- Support horizontal scaling during peak traffic.
- Provide monitoring, logging and tracing for critical transactions.

3. Core Business Scenario

Flash Sale Scenario

- Available stock: **100 units**
- Concurrent customers: **10,000**
- All customers may click **Buy Now** at approximately the same time.
- Maximum successful sales must never exceed **100 units**.
- Customers who cannot reserve inventory must receive an appropriate failure/out-of-stock response.

Expected Traffic

- Normal traffic: approximately **10,000 requests/second**
- Flash-sale traffic: architecture should reason about up to **500,000 requests/second**.

4. Functional Requirements

FR1 – User & Product Discovery

The system shall allow customers to:

- Access the platform securely.
- Browse products.
- View product information and availability.
- Add products to their cart.
- Initiate a purchase.

FR2 – Cart Management

The system shall:

- Create and maintain customer carts.
- Add/remove products.
- Maintain cart items and quantities.
- Proceed from cart to checkout.

FR3 – Inventory Management

The system shall:

- Maintain available, reserved and sold quantities.
- Check inventory availability.
- Prevent inventory quantity from becoming negative.
- Handle simultaneous purchase requests safely.

FR4 – Inventory Reservation

The system shall:

- Temporarily reserve available inventory.
- Associate reservations with customers/orders.
- Store reservation expiry time.
- Prevent duplicate reservations.
- Release expired or failed reservations.

FR5 – Payment Processing

The system shall:

- Initiate payment securely.
- Handle successful payments.
- Handle failed payments.
- Handle payment timeouts.
- Prevent duplicate payment transactions using idempotency.

FR6 – Order Management

The system shall:

- Create orders after successful purchase processing.
- Maintain valid order states.
- Handle cancellation and failure scenarios.
- Continue/recover order processing after temporary service failures.

FR7 – Fulfilment & Shipment

The system shall:

- Pass confirmed orders to fulfilment.
- Track shipment status.
- Maintain delivery-related information.

FR8 – Notifications

The system shall generate appropriate notifications for important events such as:

- Order confirmation
- Payment failure
- Order/shipment updates

FR9 – Duplicate Request Handling

The system shall use an **idempotency mechanism** so that repeated purchase or payment requests do not create duplicate business transactions.

FR10 – Failure Recovery

The system shall provide recovery mechanisms for:

- Payment failure
- Payment timeout
- Order service failure

- Database/service failure
- Failed asynchronous messages
- External dependency failure

5. Inventory Lifecycle

The inventory/reservation lifecycle shall support:

AVAILABLE → RESERVED → PAYMENT_PENDING → CONFIRMED → SOLD

Failure paths:

RESERVED → PAYMENT_FAILED → RELEASED

RESERVED → TIMEOUT → RELEASED

The design must guarantee that successful reservations never exceed available inventory.

6. Order Lifecycle

The order lifecycle shall support:

CREATED → PAYMENT_PENDING → CONFIRMED → PROCESSING → SHIPPED → OUT_FOR_DELIVERY → DELIVERED

Appropriate failure, cancellation and recovery states shall also be considered.

7. Non-Functional Requirements

7.1 Scalability

- Support approximately **10,000 requests/second** during normal traffic.
- Reason about up to **500,000 requests/second** during flash-sale traffic.
- Support horizontal scaling.
- Avoid single points of bottleneck where possible.

7.2 Performance

- Handle high concurrent purchase traffic.
- Control contention at the inventory reservation point.
- Use caching, queueing and load distribution where appropriate.
- Maintain predictable behaviour during traffic spikes.

7.3 Consistency

- Inventory must never become negative.
- Successful sales must never exceed available stock.
- Reservations must be uniquely identifiable.
- Duplicate requests must not create duplicate transactions.

7.4 Availability

- The system should remain available during traffic spikes.
- Non-critical asynchronous operations should not unnecessarily block the purchase workflow.
- Service failures should have defined recovery paths.

7.5 Reliability

- Payment failures must not leave inventory permanently reserved.
- Payment timeouts must be safely retried or reconciled.
- Payment success followed by Order Service failure must have a recovery mechanism.
- Failed asynchronous messages must have appropriate handling.

7.6 Security

The system shall consider:

- Authentication and authorization
- HTTPS
- Rate limiting
- Abuse protection
- Input validation
- Secure payment-data handling
- Audit logging
- Secrets management

7.7 Observability

The system shall provide:

- Request-rate monitoring
- Latency monitoring
- Error-rate monitoring
- Inventory reservation failure metrics
- Payment failure metrics
- Order conversion metrics
- Structured logs

- Distributed tracing
- Alerts for inventory inconsistency, payment failures and queue backlog

8. Assumptions

1. The flash-sale product initially has **100 available units**.
2. Up to **10,000 customers** may attempt to purchase simultaneously.
3. Inventory is a critical consistency boundary.
4. Payment processing depends on an external payment gateway.
5. External services may experience delays, timeouts or failures.
6. Reservations require an expiry mechanism.
7. Repeated requests may occur and must be handled idempotently.
8. The architecture must support horizontal scaling.
9. Asynchronous messaging may be used for operations that do not need to block the critical purchase path.
10. Every team member must understand and defend the architecture.

9. Strict Guarantees vs Targets

Strict Guarantees

These must never be violated:

- No inventory overselling.
- Inventory must never become negative.
- No duplicate successful payment transaction for the same request.
- No duplicate successful reservation for the same idempotent request.
- Correct order state transitions.
- Expired/failed reservations must be released.
- Critical transactions must be traceable.

Performance / Operational Targets

These are system targets:

- Support approximately 10,000 requests/second normally.
- Reason about up to 500,000 requests/second during extreme flash-sale traffic.
- Horizontally scale services during traffic spikes.
- Recover gracefully from temporary failures.
- Maintain acceptable latency and error rates under load.

10. Critical Dependencies

The system depends on:

- Product Service
- Cart Service
- Inventory/Reservation Service
- Checkout Service
- Payment Gateway
- Order Service
- Shipment/Fulfilment Service
- Notification Service
- Database
- Cache
- Message Queue/Broker
- Monitoring, Logging and Distributed Tracing infrastructure

11. Major Risks / Failure Scenarios

The design must address:

1. 10,000 customers competing for 100 units.
2. Two customers requesting the last available unit simultaneously.
3. Duplicate Buy requests.
4. Reservation expiry.
5. Payment failure.
6. Payment timeout.
7. Duplicate payment request.
8. Payment succeeds but Order Service is unavailable.
9. Database failure.
10. Payment gateway failure.
11. Message delivery failure.
12. Traffic increasing significantly beyond expected levels.

12. Key Design Questions

Our architecture must answer:

- Where exactly is inventory consistency guaranteed?
- How are simultaneous inventory requests controlled?
- How do we guarantee that only 100 customers can successfully purchase 100 units?
- How are duplicate requests handled?
- When is inventory reserved?
- When is inventory released?
- What happens when payment fails?

- What happens when payment succeeds but Order Service fails?
- Which operations are synchronous?
- Which operations are asynchronous?
- Where should requests be throttled, queued or rejected?
- Which component is the likely bottleneck?
- How does the system scale horizontally?
- How does the system recover from database or external-service failures?

13. Initial Success Criteria

The design will be considered successful if:

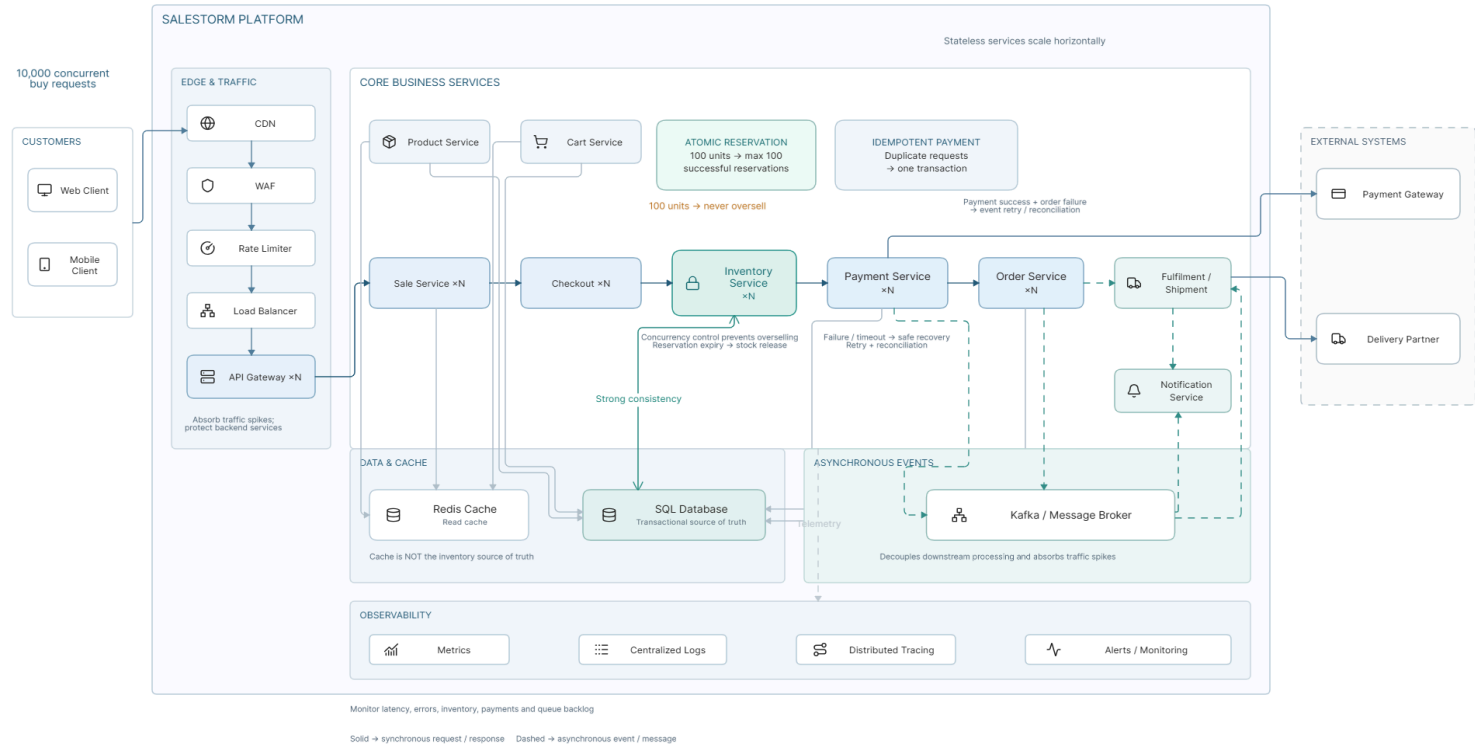
- 10,000 concurrent purchase attempts can be controlled.
- No more than 100 units can be successfully sold.
- Duplicate requests do not create duplicate transactions.
- Expired/failed reservations are released.
- Payment failures are handled safely.
- Payment success followed by Order Service failure has a recovery path.
- The architecture can scale horizontally.
- Critical failures are observable and recoverable.
- All major design decisions can be technically justified.

HIGH LEVEL DESIGN

SALESTORM — High-Scale Flash Sale System

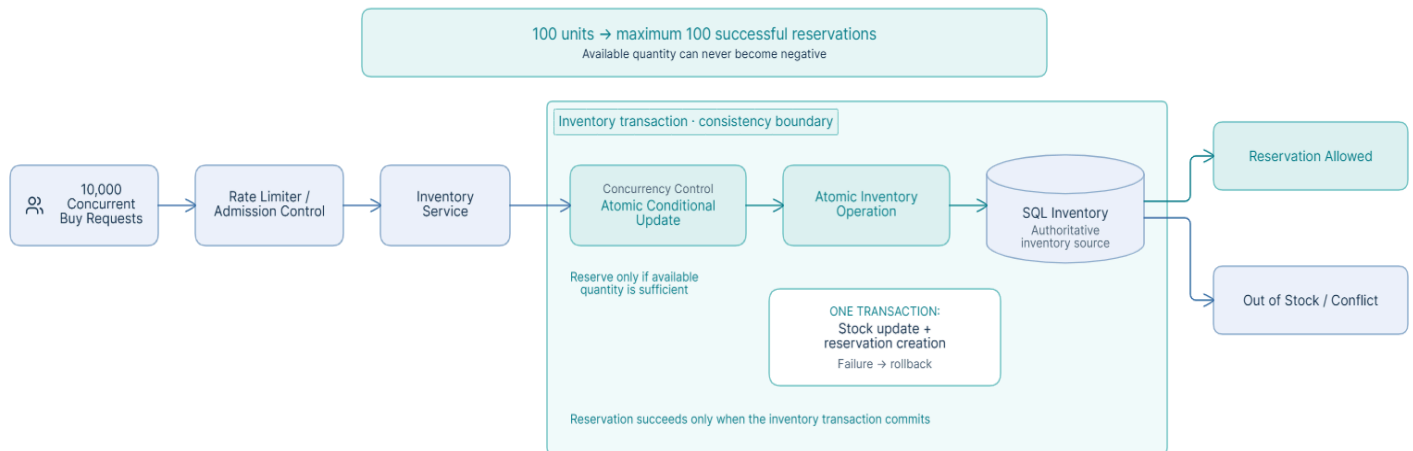
10,000 Concurrent Requests | 100 Units | Zero Overselling

Stateless services + strong inventory consistency + asynchronous downstream processing



Phase II: High Concurrency and Inventory Reservation Design

CONCURRENCY CONTROL DESIGN



Concurrency options

Pessimistic Locking

- Strong consistency
- Higher lock contention

Optimistic Locking

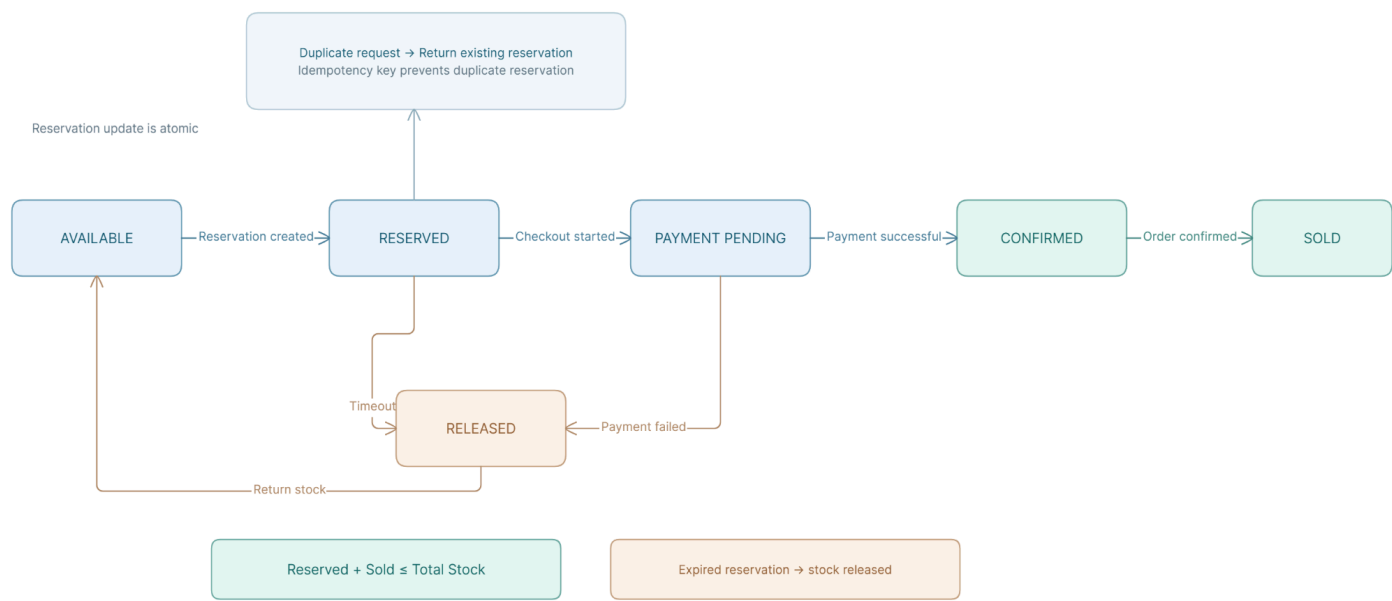
- Detects concurrent conflicts
- Retries may increase under heavy contention

Selected Approach

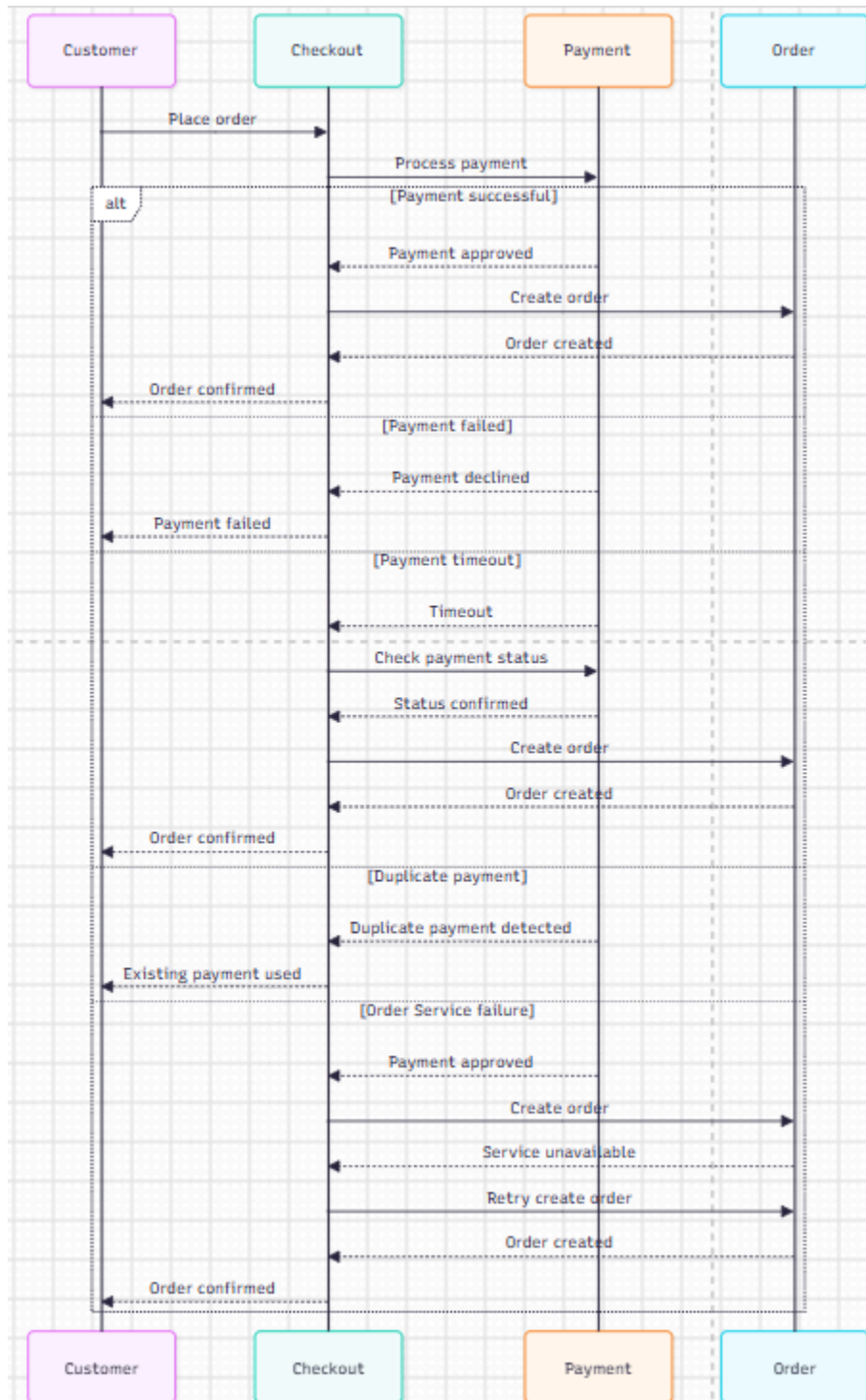
- Atomic conditional inventory update
- Deterministic success/failure
- Prevents overselling

Admission control reduces contention before the inventory consistency boundary.

INVENTORY RESERVATION DESIGN

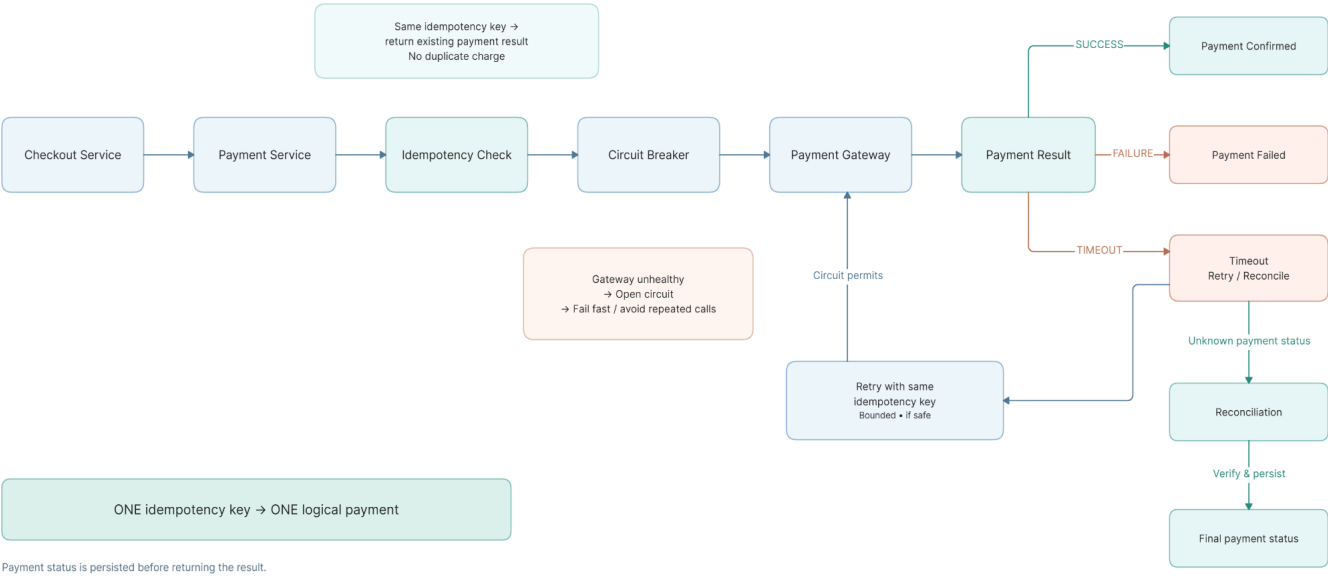


Phase III: Payment Reliability and Order Management Design



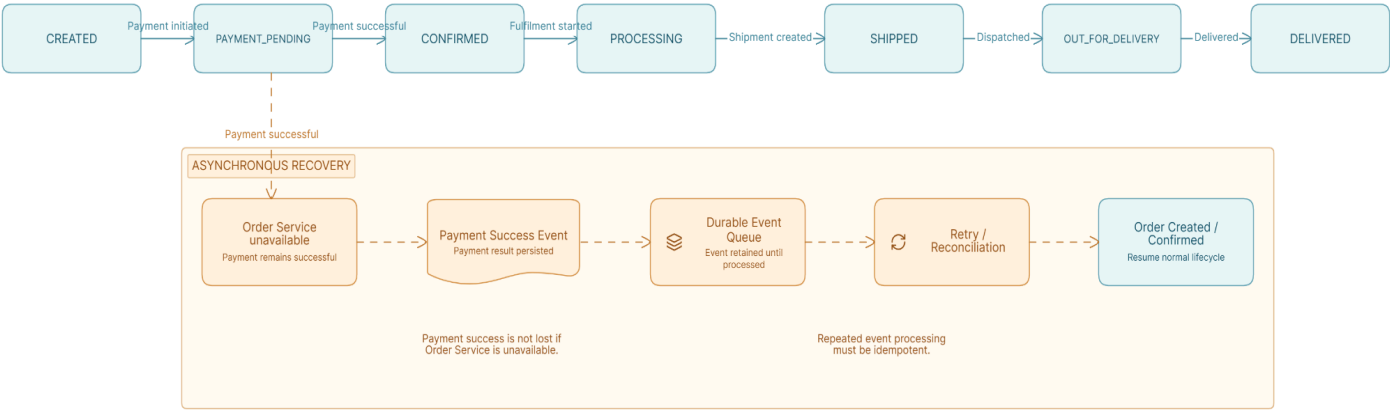
PAYMENT RELIABILITY DESIGN

Idempotent Payment | Safe Retry | Timeout | Circuit Breaker



ORDER MANAGEMENT DESIGN

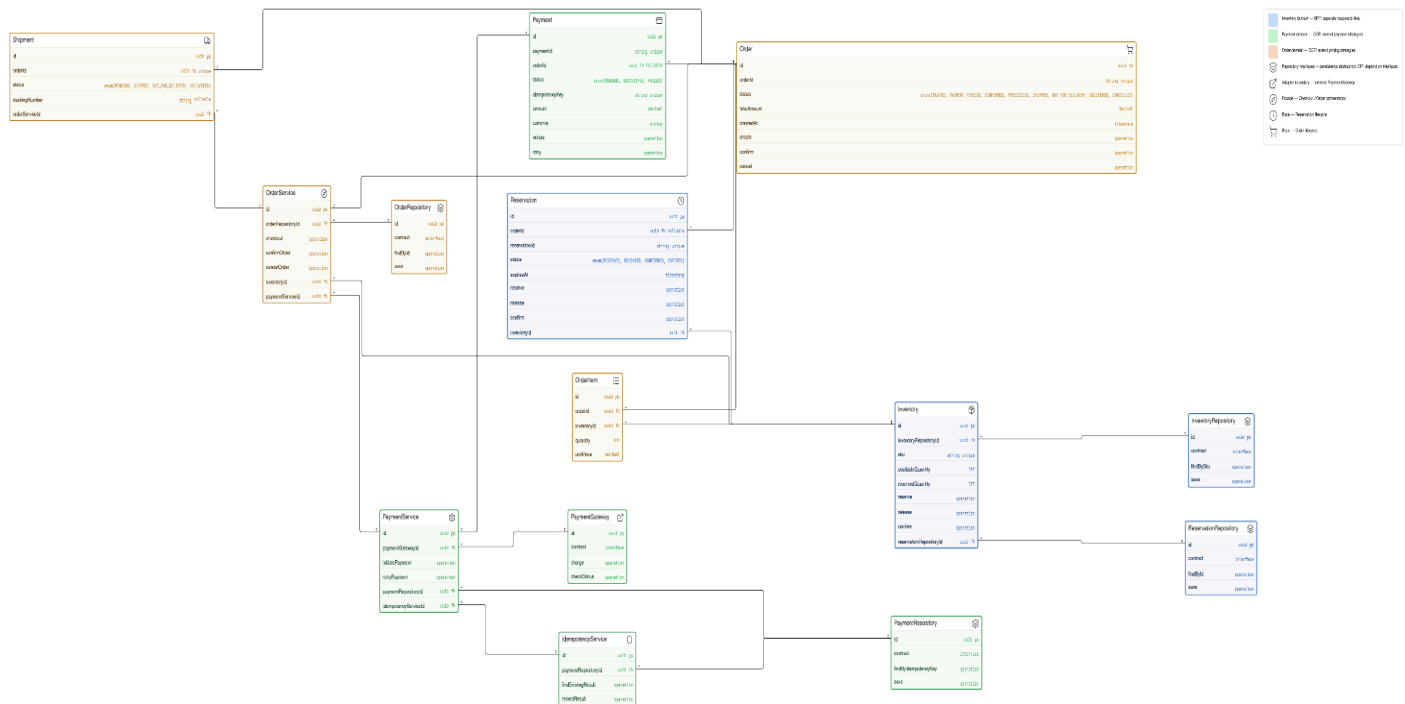
Reliable Order Creation After Payment



Order creation failure → retry / reconcile

Recovery uses the existing successful payment — no second payment transaction.

LOW LEVEL DESIGN



SOLID PRINCIPLES AND DESIGN PATTERNS

Principles and patterns applied to real SALESTORM components

